VINSUP SKILL ACADEMY

148 A,B, Gopalaswamy Kovil Street, Ganapathy, Coimbatore - 641006

PROJECT DOCUMENTATION

Hero Retail Sales Analytics

Using Excel, SQL, Power BI and Python

Domain: Retail, Sales & Inventory Analytics

Project Done By: S.Seshang

1. Project Overview

This document reports on the completion of the Hero Retail Sales Analytics project, an end-to-end analytics case study covering data cleaning, relational database design and querying, exploratory data analysis, customer segmentation, and dashboard reporting. The project followed a four-phase pipeline — Excel, SQL, Python, and Power BI — moving raw transactional retail data through cleaning, storage, analysis, and visualization stages.

Domain: Retail, Sales & Inventory Analytics

1.1 Problem Statement

In a competitive retail market, businesses need to track sales performance, understand customer behaviour, and identify growth opportunities. This project builds an end-to-end analytics solution that generates insights from raw transactional data using a multi-tool approach spanning Excel, SQL, Python, and Power BI.

1.2 Business Use Cases Addressed

- Analyse sales performance across different stores and staff
- Identify high-value customers and buying behaviour patterns
- Perform customer segmentation for targeted marketing
- Monitor inventory levels and product performance
- Deliver an interactive Power BI dashboard for management insights

1.3 Skills Demonstrated

- Data cleaning and preprocessing using Excel
- Data storage, querying, and transformation using SQL
- Data analysis and visualization using Python (Pandas, NumPy, Matplotlib/Seaborn)
- Customer segmentation using RFM (Recency, Frequency, Monetary) analysis
- Business intelligence and reporting using Power BI
- End-to-end data pipeline integration across four tools

2. Dataset Overview

The retail dataset ("Hero Cycles") is organised as nine related tables covering customers, orders, order line items, products, brands, categories, stock, stores, and staff. The final row counts in the cleaned working files were:

Table	Description	Records
customers	Buyer information including contact and location	1,445
orders	Transaction log with staff and store references	1,615
order_items	Detailed line items per order (quantity, price, discount, sales)	4,722
products	Item details with category, brand, model year and price	321
brands	Brand metadata	9
categories	Product category metadata	7
stocks	Inventory quantities by store and product	939
stores	Store master data (address, contact)	3
staffs	Staff master data and reporting structure	10

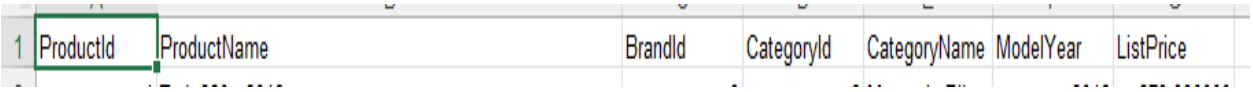
Key fields used across the pipeline include order_id, customer_id, store_id, staff_id, product_id, order_date, order_status, quantity, list_price, discount, and total_sales — matching the relational structure defined in the ERD supplied for the project.

3. Phase 1 – Excel: Data Cleaning & Preparation

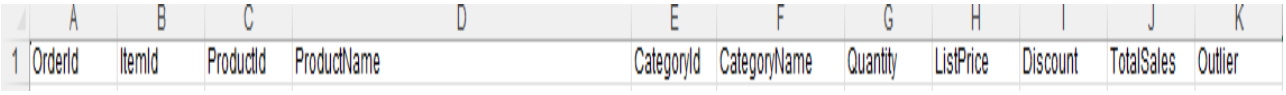
Each of the nine source sheets (brands, categories, customers, order_items, orders, products, staffs, stocks, stores) was cleaned and standardised in Excel before being loaded into SQL. The following tasks were carried out:

1. Standardize Column Headers – renamed columns consistently (e.g. CustomerId, OrderId, ProductId) across all sheets.

	A	B	C	D	E	F	G	H	I	J	K
1	CustomerId	Name	LastName	Phone	email	Street	City	State	ZipCode		

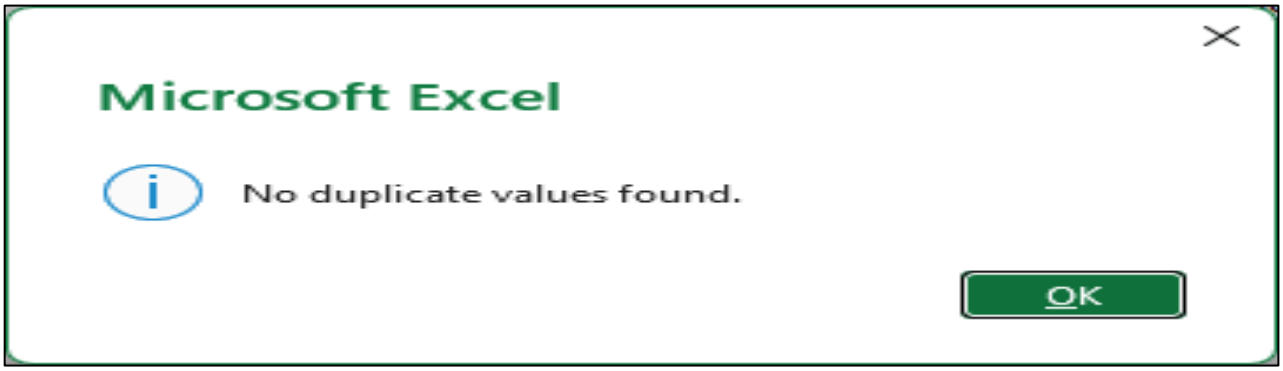


1	ProductId	ProductName	BrandId	CategoryId	CategoryName	ModelYear	ListPrice

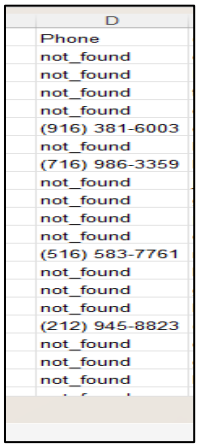


	A	B	C	D	E	F	G	H	I	J	K
1	OrderId	ItemId	ProductId	ProductName	CategoryId	CategoryName	Quantity	ListPrice	Discount	TotalSales	Outlier

Remove Duplicates – checked customers and order_items for duplicate records.

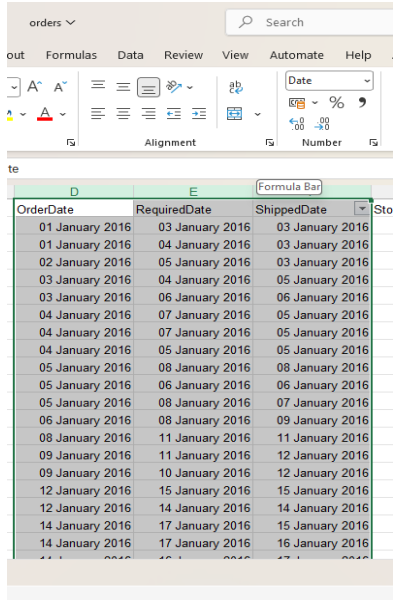


2. Handle Missing Values – filtered for blanks/nulls; missing ZIP codes and phone numbers flagged and filled with placeholders.



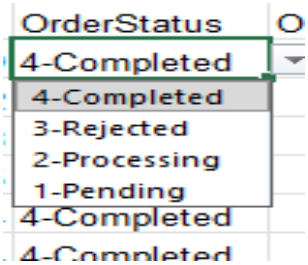
D
Phone
not_found
not_found
not_found
not_found
(916) 381-6003
not_found
(716) 986-3359
not_found
not_found
not_found
not_found
(516) 583-7761
not_found
not_found
not_found
(212) 945-8823
not_found
not_found
not_found

3. Data Type Conversion – converted OrderDate, RequiredDate and ShippedDate to proper date format; ensured Quantity, ListPrice and Discount were numeric.

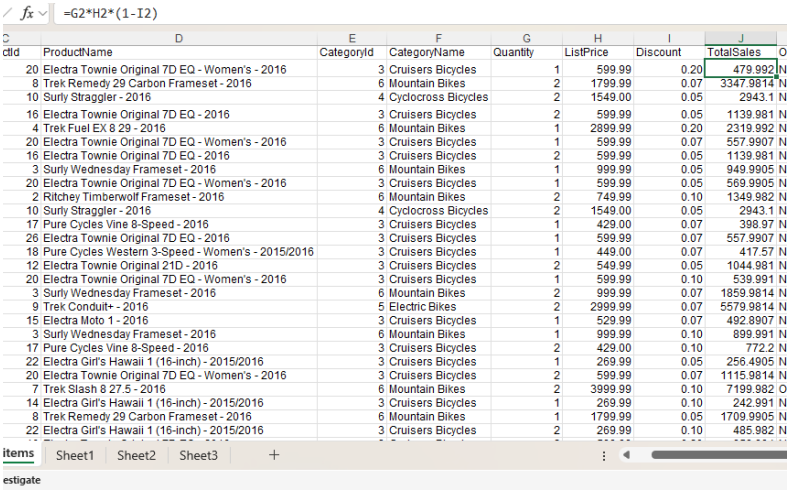


OrderDate	RequiredDate	ShippedDate
01 January 2016	03 January 2016	03 January 2016
01 January 2016	04 January 2016	03 January 2016
02 January 2016	05 January 2016	03 January 2016
03 January 2016	04 January 2016	05 January 2016
03 January 2016	06 January 2016	06 January 2016
04 January 2016	07 January 2016	05 January 2016
04 January 2016	07 January 2016	05 January 2016
04 January 2016	05 January 2016	05 January 2016
05 January 2016	08 January 2016	08 January 2016
05 January 2016	06 January 2016	06 January 2016
05 January 2016	08 January 2016	07 January 2016
06 January 2016	08 January 2016	09 January 2016
08 January 2016	11 January 2016	11 January 2016
09 January 2016	11 January 2016	12 January 2016
09 January 2016	10 January 2016	12 January 2016
12 January 2016	15 January 2016	15 January 2016
12 January 2016	14 January 2016	14 January 2016
14 January 2016	17 January 2016	15 January 2016
14 January 2016	17 January 2016	16 January 2016

4. Data Validation – applied dropdown validation on OrderStatus (Pending, Shipped, Delivered / Cancelled) for consistency.



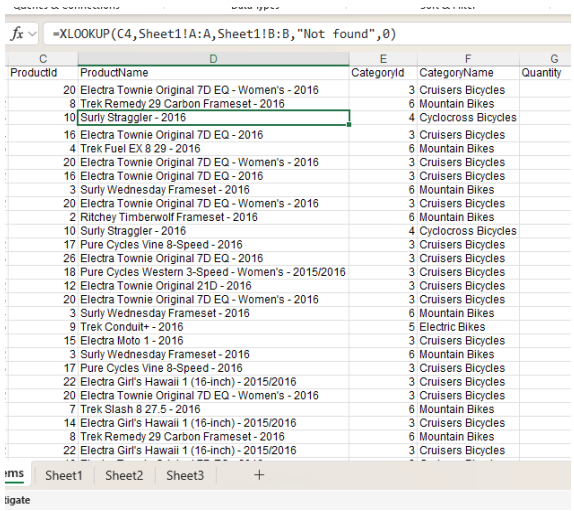
5. Derived Columns – added TotalSales in order_items, calculated as (ListPrice × Quantity) − Discount, plus an Outlier flag column.



=G2*H2*(1-I2)

ctId	ProductName	CategoryId	CategoryName	Quantity	ListPrice	Discount	TotalSales	O
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.20	479.992	N
8	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	2	1799.99	0.07	3347.9814	N
10	Surly Straggler - 2016	4	Cyclocross Bicycles	2	1549.00	0.05	2943.1	N
16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	2	599.99	0.05	1139.981	N
4	Trek Fuel EX 8 29 - 2016	6	Mountain Bikes	1	2899.99	0.20	2319.992	N
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.07	557.9907	N
16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	2	599.99	0.05	1139.981	N
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	1	999.99	0.05	949.9905	N
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.05	569.9905	N
2	Ritchey Timberwolf Frameset - 2016	6	Mountain Bikes	2	749.99	0.10	1349.982	N
10	Surly Straggler - 2016	4	Cyclocross Bicycles	2	1549.00	0.05	2943.1	N
17	Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	1	429.00	0.07	398.97	N
26	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	1	599.99	0.07	557.9907	N
18	Pure Cycles Western 3-Speed - Women's - 2015/2016	3	Cruisers Bicycles	1	449.00	0.07	417.57	N
12	Electra Townie Original 21D - 2016	3	Cruisers Bicycles	2	549.99	0.05	1044.981	N
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.10	539.991	N
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	2	999.99	0.07	1859.9814	N
9	Trek Conduit+ - 2016	5	Electric Bikes	2	2999.99	0.07	5579.9814	N
15	Electra Moto 1 - 2016	3	Cruisers Bicycles	1	529.99	0.07	492.8907	N
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	1	999.99	0.10	899.991	N
17	Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	2	429.00	0.10	772.2	N
22	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	1	269.99	0.05	256.4905	N
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	2	599.99	0.07	1115.9814	N
7	Trek Slash 8 27.5 - 2016	6	Mountain Bikes	2	3999.99	0.10	7199.982	O
14	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	1	269.99	0.10	242.991	N
8	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	1	1799.99	0.05	1709.9905	N
22	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	2	269.99	0.10	485.982	N

6. Merge Lookup Data – used VLOOKUP/XLOOKUP to bring ProductName and CategoryName into order_items from the products table.

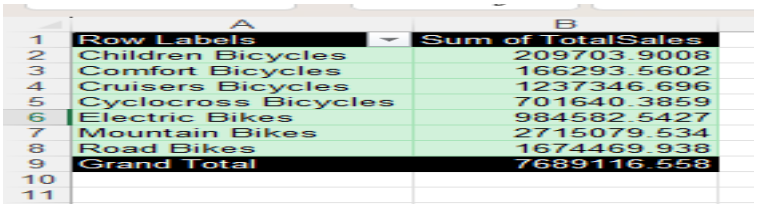


=XLOOKUP(C4,Sheet1!A:A,Sheet1!B:B,"Not found",0)

C	D	E	F	G
ProductId	ProductName	CategoryId	CategoryName	Quantity
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	
8	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	
10	Surly Straggler - 2016	4	Cyclocross Bicycles	
16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	
4	Trek Fuel EX 8 29 - 2016	6	Mountain Bikes	
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	
16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	
2	Ritchey Timberwolf Frameset - 2016	6	Mountain Bikes	
10	Surly Straggler - 2016	4	Cyclocross Bicycles	
17	Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	
26	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	
18	Pure Cycles Western 3-Speed - Women's - 2015/2016	3	Cruisers Bicycles	
12	Electra Townie Original 21D - 2016	3	Cruisers Bicycles	
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	
9	Trek Conduit+ - 2016	5	Electric Bikes	
15	Electra Moto 1 - 2016	3	Cruisers Bicycles	
3	Surly Wednesday Frameset - 2016	6	Mountain Bikes	
17	Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	
22	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	
20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	
7	Trek Slash 8 27.5 - 2016	6	Mountain Bikes	
14	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	
8	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	
22	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	

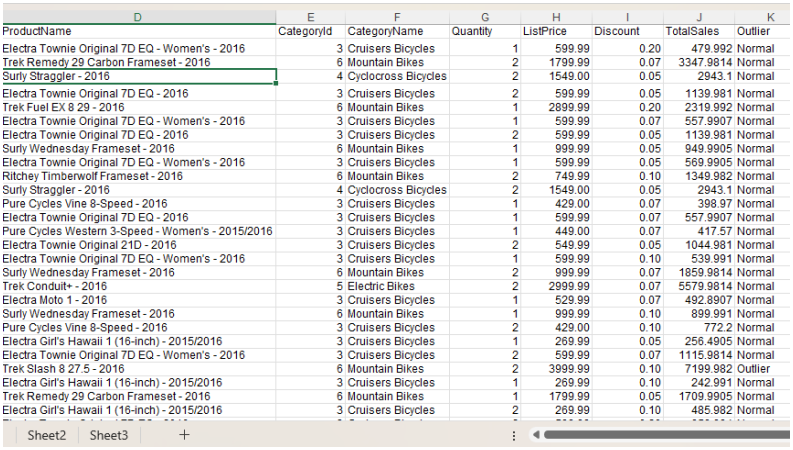
Sheet1 Sheet2 Sheet3

7. Pivot Table – built a pivot summarising total sales by product category.



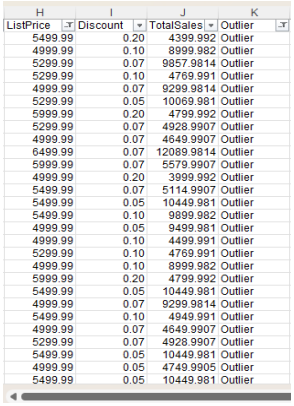
Row Labels	Sum of TotalSales
Children Bicycles	209703.9008
Comfort Bicycles	166293.5602
Cruisers Bicycles	1237346.696
Cyclocross Bicycles	701640.3859
Electric Bikes	984582.5427
Mountain Bikes	2715079.534
Road Bikes	1674469.938
Grand Total	7689116.558

8. Outlier Review – sorted and filtered list_price to identify unusually high/low priced products.



D	E	F	G	H	I	J	K
ProductName	CategoryId	CategoryName	Quantity	ListPrice	Discount	TotalSales	Outlier
Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.20	479.992	Normal
Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	2	1799.99	0.07	3347.9814	Normal
Surly Straggler - 2016	4	Cyclocross Bicycles	2	1549.00	0.05	2943.1	Normal
Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	2	599.99	0.05	1139.981	Normal
Trek Fuel EX 8 29 - 2016	6	Mountain Bikes	1	2899.99	0.20	2319.992	Normal
Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.07	557.9907	Normal
Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	2	599.99	0.05	1139.981	Normal
Surly Wednesday Frameset - 2016	6	Mountain Bikes	1	999.99	0.05	949.9905	Normal
Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.05	569.9905	Normal
Ritchey Timberwolf Frameset - 2016	6	Mountain Bikes	2	749.99	0.10	1349.982	Normal
Surly Straggler - 2016	4	Cyclocross Bicycles	2	1549.00	0.05	2943.1	Normal
Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	1	429.00	0.07	398.97	Normal
Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycles	1	599.99	0.07	557.9907	Normal
Pure Cycles Western 3-Speed - Women's - 2015/2016	3	Cruisers Bicycles	1	449.00	0.07	417.57	Normal
Electra Townie Original 21D - 2016	3	Cruisers Bicycles	2	549.99	0.05	1044.981	Normal
Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	1	599.99	0.10	539.991	Normal
Surly Wednesday Frameset - 2016	6	Mountain Bikes	2	999.99	0.07	1859.9814	Normal
Trek Conduit+ - 2016	5	Electric Bikes	2	2999.99	0.07	5579.9814	Normal
Electra Moto 1 - 2016	3	Cruisers Bicycles	1	529.99	0.07	492.8907	Normal
Surly Wednesday Frameset - 2016	6	Mountain Bikes	1	999.99	0.10	899.991	Normal
Pure Cycles Vine 8-Speed - 2016	3	Cruisers Bicycles	2	429.00	0.10	772.2	Normal
Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	1	269.99	0.05	256.4905	Normal
Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycles	2	599.99	0.07	1115.9814	Normal
Trek Slash 8 27.5 - 2016	6	Mountain Bikes	2	3999.99	0.10	7199.982	Outlier
Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	1	269.99	0.10	242.991	Normal
Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bikes	1	1799.99	0.05	1709.9905	Normal
Electra Girl's Hawaii 1 (16-inch) - 2015/2016	3	Cruisers Bicycles	2	269.99	0.10	485.982	Normal

Sheet2 Sheet3



H	I	J	K
ListPrice	Discount	TotalSales	Outlier
5499.99	0.20	4399.992	Outlier
4999.99	0.10	8999.982	Outlier
5299.99	0.07	9857.9814	Outlier
5299.99	0.10	4769.991	Outlier
4999.99	0.07	9299.9814	Outlier
5299.99	0.05	10069.981	Outlier
5999.99	0.20	4799.992	Outlier
5299.99	0.07	4928.9907	Outlier
4999.99	0.07	4649.9907	Outlier
6499.99	0.07	12089.9814	Outlier
5999.99	0.07	5579.9907	Outlier
4999.99	0.20	3999.992	Outlier
5499.99	0.07	5114.9907	Outlier
5499.99	0.05	10449.981	Outlier
5499.99	0.10	9899.982	Outlier
4999.99	0.05	9499.981	Outlier
4999.99	0.10	4499.991	Outlier
5299.99	0.10	4769.991	Outlier
4999.99	0.10	8999.982	Outlier
5999.99	0.20	4799.992	Outlier
5499.99	0.05	10449.981	Outlier
4999.99	0.07	9299.9814	Outlier
5499.99	0.10	4949.991	Outlier
4999.99	0.07	4649.9907	Outlier
5299.99	0.07	4928.9907	Outlier
5499.99	0.05	10449.981	Outlier
4999.99	0.05	4749.9905	Outlier
5499.99	0.05	10449.981	Outlier

OUTLIERS	
Q1	429
Q3	1549
IPR	1120
LOWER	-1251
UPPER	3229

9. Final CSV/XLSX Export – saved cleaned sheets (brands.xlsx, categories.xlsx, customers.xlsx, order_items.xlsx, orders.xlsx, products.xlsx, staffs.xlsx, stocks.xlsx, stores.xlsx) ready for SQL import.

Output: nine cleaned workbook files, each matching the ERD structure and ready for direct import into the SQL database.

4. Phase 2 – SQL: Database Management & Querying

A relational database named hero was created in MySQL and populated from the cleaned Excel exports. Ten SQL tasks were completed, covering table creation, joins, aggregation, segmentation logic, and a staging table for the machine-learning output.

4.1 Database Setup

- Created the hero database and defined tables for brands, categories, customers, orders, order_items, products, staffs, stocks and stores, replicating the ERD structure.
- Imported the cleaned Excel/CSV files into their corresponding SQL tables.

4.2 Analytical Queries Executed

#	Query	Purpose
1	Inner Join for Order Details	Joined orders, order_items and products to produce a full order line-item view.
2	Total Sales by Store	Grouped order_items by StoreId and summed TotalSales per store.
3	Top 5 Selling Products	Ranked products by number of units sold using ORDER BY and LIMIT.
4	Customer Purchase Summary	Returned total orders placed, total items purchased and total revenue per customer.
5	Segment Customers by Total Spend	Classified order lines into Low / Medium / High spending brackets using CASE WHEN logic.
6	Staff Performance Analysis	Aggregated total orders and total revenue handled by each staff member, ranked by revenue.

#	Query	Purpose
7	Stock Alert Query	Summed stock quantity per product across stores to flag low-stock items.
8	Create Final Segmentation Table	Created an empty customer_segments (Recency, Frequency, Monetary) table to be populated from the Python RFM output.

These queries directly answer the core business questions: which stores and staff generate the most revenue, which products sell best, which customers are most valuable, and which products are at risk of stocking out.

QUERIES USED:

```
Create database hero;

-- 1.Create Tables Based on ERD

-- 2. Import CSVs into SQL

select  * from hero.brands;

select * from hero.categories;

select * from hero.customers;

select * from hero.order_items;

select * from hero.orders;

select * from hero.products;

select * from hero.staffs;

select * from hero.stocks;

select * from hero.stores;

select * from hero.consumer_segments;
```

-- 3. Inner Join for Order Details

```
select o.*,ot.*,p.* from  hero.orders as o

join hero.order_items as ot

on o.OrderId=ot.OrderId

join hero.products as p

on ot.ProductId=p.ProductId;
```

26 group by o.StoreId;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

OrderId	CustomerId	OrderStatus	OrderDate	RequiredDate	ShippedDate	StoreId	StaffId	OrderId	ItemId	ProductId	ProductName	CategoryId	CategoryNam
1	259	4-Completed	2016-01-01	2016-01-03	2016-01-03	1	2	1	1	20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycl
1	259	4-Completed	2016-01-01	2016-01-03	2016-01-03	1	2	1	2	8	Trek Remedy 29 Carbon Frameset - 2016	6	Mountain Bike
1	259	4-Completed	2016-01-01	2016-01-03	2016-01-03	1	2	1	3	10	Surly Straggler - 2016	4	Cyclocross Bic
1	259	4-Completed	2016-01-01	2016-01-03	2016-01-03	1	2	1	4	16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycl
1	259	4-Completed	2016-01-01	2016-01-03	2016-01-03	1	2	1	5	4	Trek Fuel EX 8 29 - 2016	6	Mountain Bike
2	1212	4-Completed	2016-01-01	2016-01-04	2016-01-03	2	6	2	1	20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycl
2	1212	4-Completed	2016-01-01	2016-01-04	2016-01-03	2	6	2	2	16	Electra Townie Original 7D EQ - 2016	3	Cruisers Bicycl
3	523	4-Completed	2016-01-02	2016-01-05	2016-01-03	2	7	3	1	3	Surly Wednesday Frameset - 2016	6	Mountain Bike
3	523	4-Completed	2016-01-02	2016-01-05	2016-01-03	2	7	3	2	20	Electra Townie Original 7D EQ - Women's - 2016	3	Cruisers Bicycl
4	175	4-Completed	2016-01-03	2016-01-04	2016-01-05	1	3	4	1	2	Ritchey Timberwolf Frameset - 2016	6	Mountain Bike

Result 1 × Read Only

-- 4. Total Sales by Store

```
select o.StoreId,sum(ot.TotalSales) as Total_Sales from hero.order_items as ot

join hero.orders as o

on o.OrderId=ot.OrderId

group by o.StoreId;
```

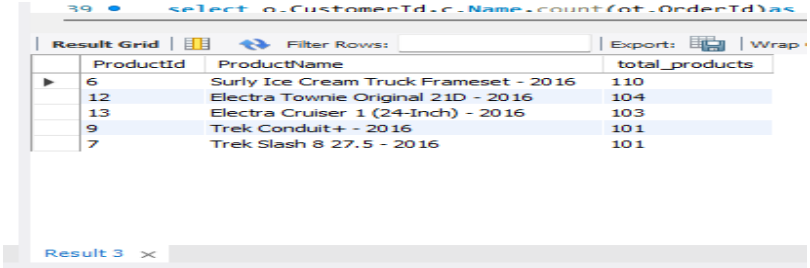
27

Result Grid | Filter Rows:

StoreId	Total_Sales
1	1605823.0364999974
2	5215751.277499983
3	867542.2436000016

-- 5. Top 5 Selling Products

```
select p.ProductId,p.ProductName,count(ot.Quantity) as total_products from hero.order_items as ot

join hero.products as p

on p.ProductId=ot.ProductId

group by p.ProductId,p.ProductName

order by total_products desc

limit 5;
```



Result Grid | Filter Rows: | Export: | Wrap

ProductId	ProductName	total_products
6	Surly Ice Cream Truck Frameset - 2016	110
12	Electra Townie Original 21D - 2016	104
13	Electra Cruiser 1 (24-Inch) - 2016	103
9	Trek Conduit+ - 2016	101
7	Trek Slash 8 27.5 - 2016	101

Result 3

-- 6. Customer Purchase Summary

select o.CustomerId,c.Name,count(ot.OrderId)as total_orders_placed,count(ot.ItemId) as total_items_purchased,sum(ot.TotalSales) as total_revenue from hero.order_items as ot

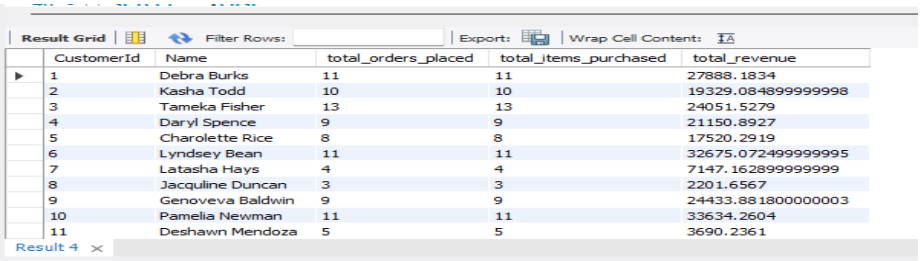
join hero.orders as o

on o.OrderId=ot.OrderId

join hero.customers as c

on c.CustomerId=o.CustomerId

group by o.CustomerId;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerId	Name	total_orders_placed	total_items_purchased	total_revenue
1	Debra Burks	11	11	27888.1834
2	Kasha Todd	10	10	19329.084899999998
3	Tameka Fisher	13	13	24051.5279
4	Daryl Spence	9	9	21150.8927
5	Charolette Rice	8	8	17520.2919
6	Lyndsey Bean	11	11	32675.072499999995
7	Latasha Hays	4	4	7147.162899999999
8	Jacquline Duncan	3	3	2201.6567
9	Genoveva Baldwin	9	9	24433.881800000003
10	Pamelia Newman	11	11	33634.2604
11	Deshawn Mendoza	5	5	3690.2361

Result 4

-- 7. Segment Customers by Total Spend

```
select  *,case
 when ot.TotalSales>4000 then "High"
when ot.TotalSales>=2000 then "Medium"
when  ot.TotalSales<2000 then "Low"
end as spending_brackets from hero.order_items as ot
left join hero.orders as o
on o.OrderId = ot.OrderId
left join hero.customers as c
on o.CustomerId = c.CustomerId;
```

57

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

ate	ShippedDate	StoreId	StaffId	CustomerId	Name	Phone	Email	Street	City	State	ZipCode	spending_brackets
	2016-01-03	1	2	259	Johnathan Velazquez	not_found	johnathan.velazquez@hotmail.com	9680 E. Somerset Street	Pleasanton	CA	94566	Low
	2016-01-03	1	2	259	Johnathan Velazquez	not_found	johnathan.velazquez@hotmail.com	9680 E. Somerset Street	Pleasanton	CA	94566	Medium
	2016-01-03	1	2	259	Johnathan Velazquez	not_found	johnathan.velazquez@hotmail.com	9680 E. Somerset Street	Pleasanton	CA	94566	Medium
	2016-01-03	1	2	259	Johnathan Velazquez	not_found	johnathan.velazquez@hotmail.com	9680 E. Somerset Street	Pleasanton	CA	94566	Low
	2016-01-03	1	2	259	Johnathan Velazquez	not_found	johnathan.velazquez@hotmail.com	9680 E. Somerset Street	Pleasanton	CA	94566	Medium
	2016-01-03	2	6	1212	Jaqueline Cummings	not_found	jaqueline.cummings@hotmail.com	478 Wrangler St.	Huntington Station	NY	11746	Low
	2016-01-03	2	6	1212	Jaqueline Cummings	not_found	jaqueline.cummings@hotmail.com	478 Wrangler St.	Huntington Station	NY	11746	Low
	2016-01-03	2	7	523	Joshua Robertson	not_found	joshua.robertson@gmail.com	81 Campfire Court	Patchogue	NY	11772	Low
	2016-01-03	2	7	523	Joshua Robertson	not_found	joshua.robertson@gmail.com	81 Campfire Court	Patchogue	NY	11772	Low
	2016-01-05	1	3	175	Nova Hess	not_found	nova.hess@msn.com	773 South Lafayette St.	Duarte	CA	91010	Low

Result 5 × Read Only

-- 8. Staff Performance Analysis

```
select o.StaffId,count(ot.OrderId) as total_orders,sum(ot.TotalSales) as total_revenue from hero.order_items as ot
join hero.orders as o
on o.OrderId=ot.OrderId
group by O.StaffId
order by total_revenue desc;
```

Result Grid | Filter Rows:

StaffId	total_orders	total_revenue
6	1615	2624120.652999998
7	1580	2591630.624499995
3	544	853287.3589000017
2	462	752535.6776000012
8	269	463918.3046000004
9	252	403623.9390000005

Result 6 ×

-- 9. Stock Alert Query

```
select p.ProductId,p.ProductName,sum(s.Quantity) as total_quantity from hero.stocks as s

join hero.products as p

on s.ProductId=p.ProductId

group by p.ProductId,p.ProductName

having total_quantity>10

order by total_quantity asc;
```

75

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductId	ProductName	total_quantity
70	Electra Amsterdam Original 3i - 2015/2017	11
105	Sun Bicycles Streamway 7 - 2017	11
51	Trek Silque SLR 8 Women's - 2017	12
82	Electra Amsterdam Original 3i Ladies' - 2017	12
235	Electra Koa 3i Ladies' - 2018	12
251	Electra Townie Commute Go! - 2018	12
302	Electra Townie Original 1 - 2018	12
8	Trek Remedy 29 Carbon Frameset - 2016	13
180	Trek Domane AL 3 Women's - 2018	14
49	Trek Domane SL 6 - 2017	16
72	Sun Bicycles Biscayne Tandem 7 - 2017	16

Result 7

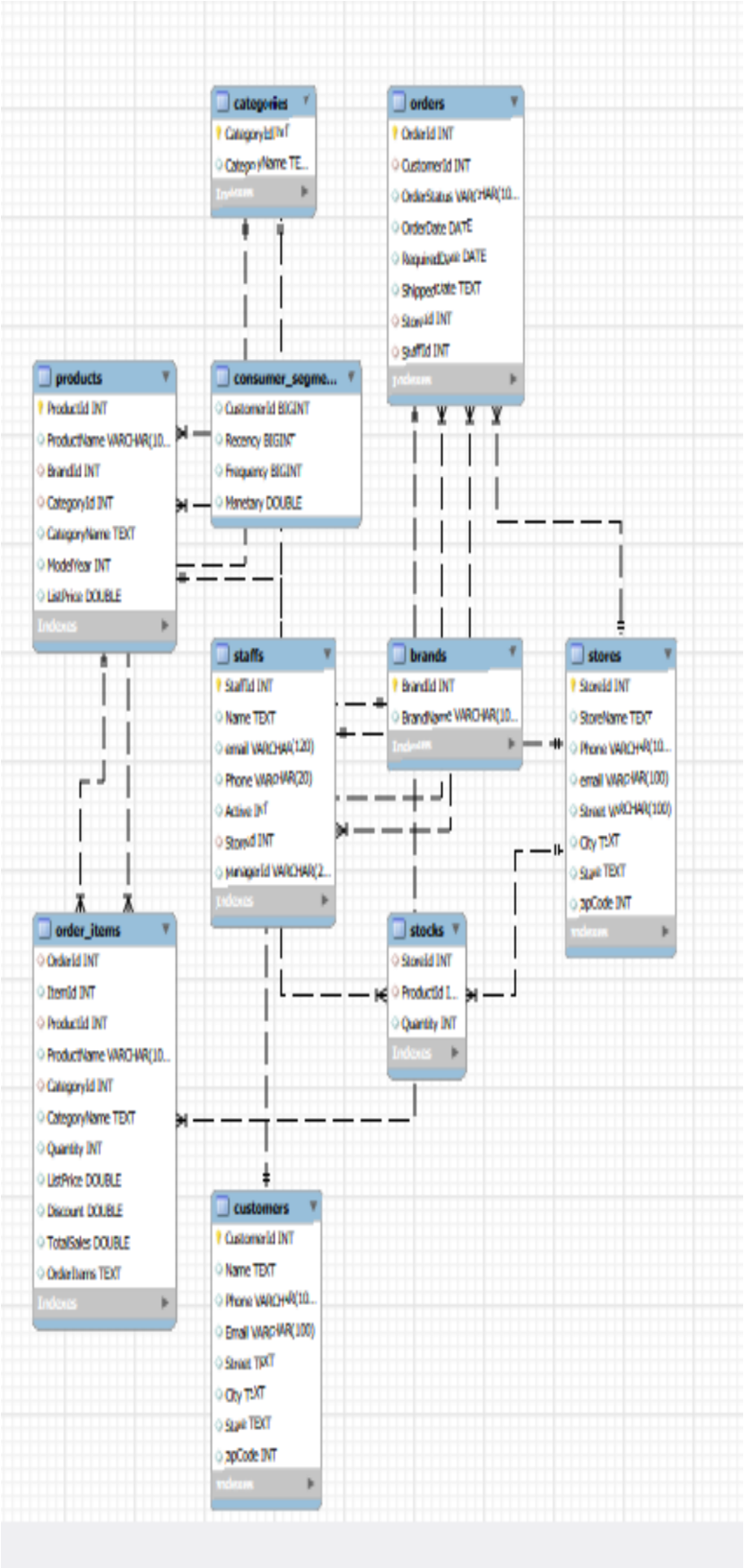
-- 10.Create Final Segmentation Table

```
create table hero.customer_segments(Recency varchar(100),Frequency varchar(100),Monetay
varchar(100));


select * from hero.customer_segments;
```

Result Grid | Filter Rows: | Export: | Wra

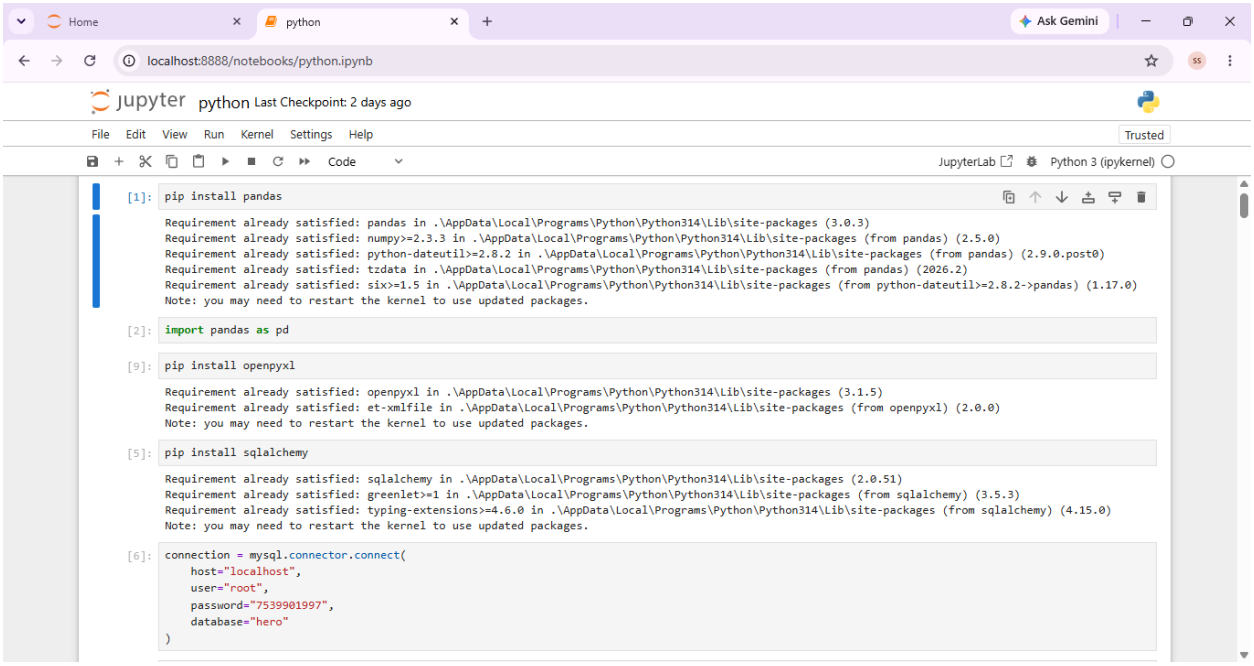
CustomerId	Recency	Frequency	Monetary
1	41	3	27888.1834
2	264	3	19329.0849
3	69	3	24051.5279
4	255	3	21150.8927
5	256	3	17520.2919

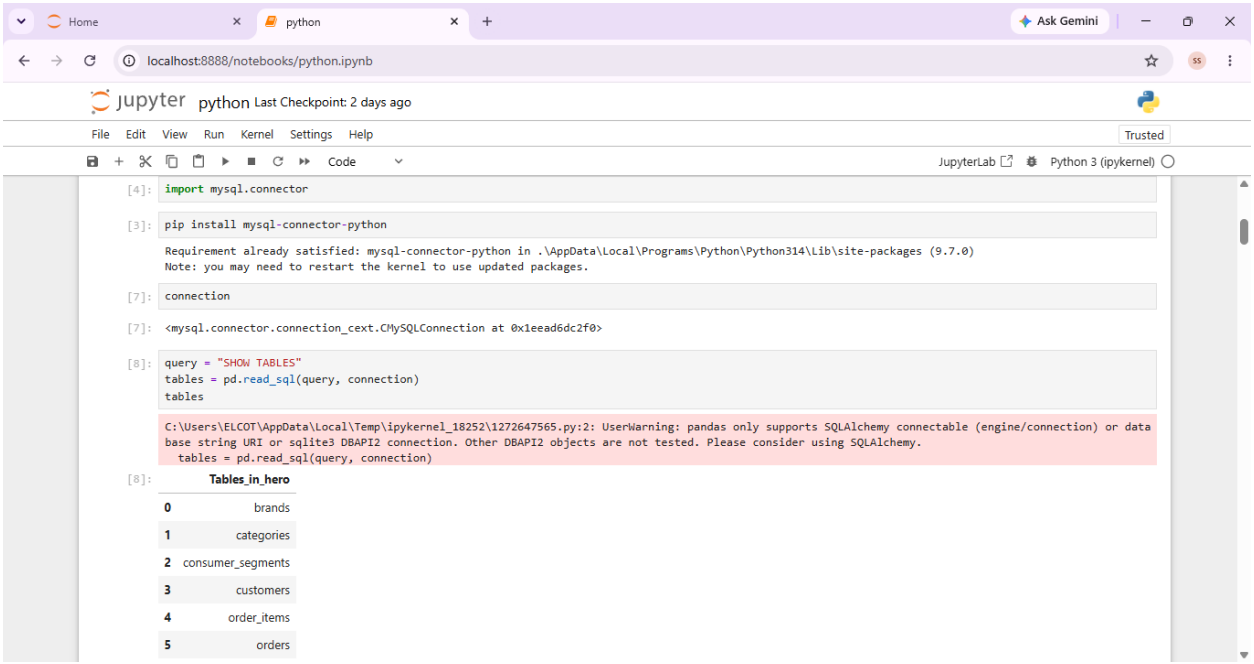
27888.1834


categories
CategoryId INT
CategoryName TE...
orders
OrderId INT
CustomerId INT
OrderStatus VARCHAR(10...
OrderDate DATE
RequiredDate DATE
ShippedDate TEXT
StoreId INT
StaffId INT
products
ProductId INT
ProductName VARCHAR(10...
BrandId INT
CategoryId INT
CategoryName TEXT
ModelYear INT
ListPrice DOUBLE
consumer_segme...
CustomerId BIGINT
Recency BIGINT
Frequency BIGINT
Monetary DOUBLE
staffs
StaffId INT
Name TEXT
email VARCHAR(120)
Phone VARCHAR(20)
Active INT
StoreId INT
managerId VARCHAR(2...
brands
BrandId INT
BrandName VARCHAR(10...
stores
StoreId INT
StoreName TEXT
Phone VARCHAR(10...
email VARCHAR(100)
Street VARCHAR(100)
City TEXT
State TEXT
zipCode INT
order_items
OrderId INT
ItemId INT
ProductId INT
ProductName VARCHAR(10...
CategoryId INT
CategoryName TEXT
Quantity INT
ListPrice DOUBLE
Discount DOUBLE
TotalSales DOUBLE
OrderItems TEXT
stocks
StoreId INT
ProductId I...
Quantity INT
customers
CustomerId INT
Name TEXT
Phone VARCHAR(10...
Email VARCHAR(100)
Street TEXT
City TEXT
State TEXT
zipCode INT

5. Phase 3 – Python & Machine Learning

A Jupyter notebook connected to the hero MySQL database (via mysql-connector-python / SQLAlchemy) to pull the orders, customers and order_items tables into Pandas DataFrames for analysis.





\

5.1 Exploratory Data Analysis (EDA)

- Used df.describe(), df.info() and df.value_counts() to profile orders (1,615 rows, 8 columns), customers (1,445 rows, 9 columns) and order_items (4,722 rows, 11 columns).

```
[15]: orders.describe()
```

[15]:

	OrderId	CustomerId	StoreId	StaffId
count	1615.000000	1615.000000	1615.000000	1615.000000
mean	808.000000	654.171517	1.892260	5.855108
std	466.354658	443.229967	0.558395	1.913899
min	1.000000	1.000000	1.000000	2.000000
25%	404.500000	237.000000	2.000000	6.000000
50%	808.000000	638.000000	2.000000	6.000000
75%	1211.500000	1041.500000	2.000000	7.000000
max	1615.000000	1445.000000	3.000000	9.000000

```
[16]: orders.info()

<class 'pandas.DataFrame'>
RangeIndex: 1615 entries, 0 to 1614
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   OrderId       1615 non-null   int64
 1   CustomerId    1615 non-null   int64
 2   OrderStatus   1615 non-null   str
 3   OrderDate     1615 non-null   object
 4   RequiredDate  1615 non-null   object
 5   ShippedDate   1615 non-null   str
 6   StoreId       1615 non-null   int64
 7   StaffId       1615 non-null   int64
dtypes: int64(4), object(2), str(2)
memory usage: 101.1+ KB
```

```
[17]: orders.value_counts()

[17]: OrderId  CustomerId  OrderStatus  OrderDate   RequiredDate  ShippedDate  StoreId  StaffId
      1        259        4-Completed  2016-01-01  2016-01-03    2016-01-03   1        2        1
      2        1212       4-Completed  2016-01-01  2016-01-04    2016-01-03   2        6        1
      3        523        4-Completed  2016-01-02  2016-01-05    2016-01-03   2        7        1
      4        175        4-Completed  2016-01-03  2016-01-04    2016-01-05   1        3        1
      5        1324       4-Completed  2016-01-03  2016-01-06    2016-01-06   2        6        1
                                                                                               ..
      1611     6          3-Rejected   2018-09-06  2018-09-06    NotFound     2        7        1
      1612     3          3-Rejected   2018-10-21  2018-10-21    NotFound     1        3        1
      1613     1          3-Rejected   2018-11-18  2018-11-18    NotFound     2        6        1
      1614     135        3-Rejected   2018-11-28  2018-11-28    NotFound     3        8        1
      1615     136        3-Rejected   2018-12-28  2018-12-28    NotFound     3        8        1
      Name: count, Length: 1615, dtype: int64
```

- Visualised the distribution of orders by status and customers by state using Matplotlib.

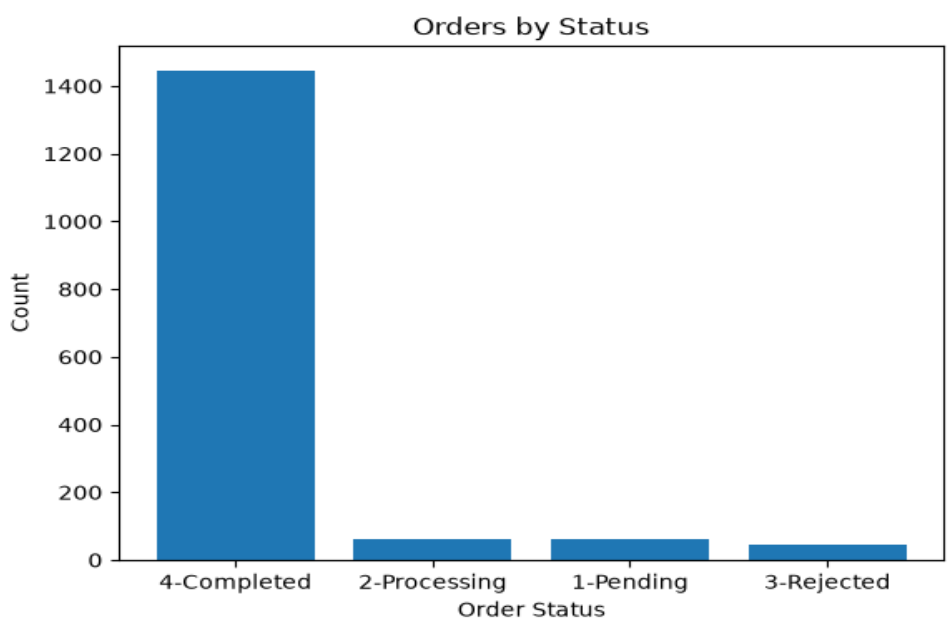


Figure 1: Distribution of orders by order status.

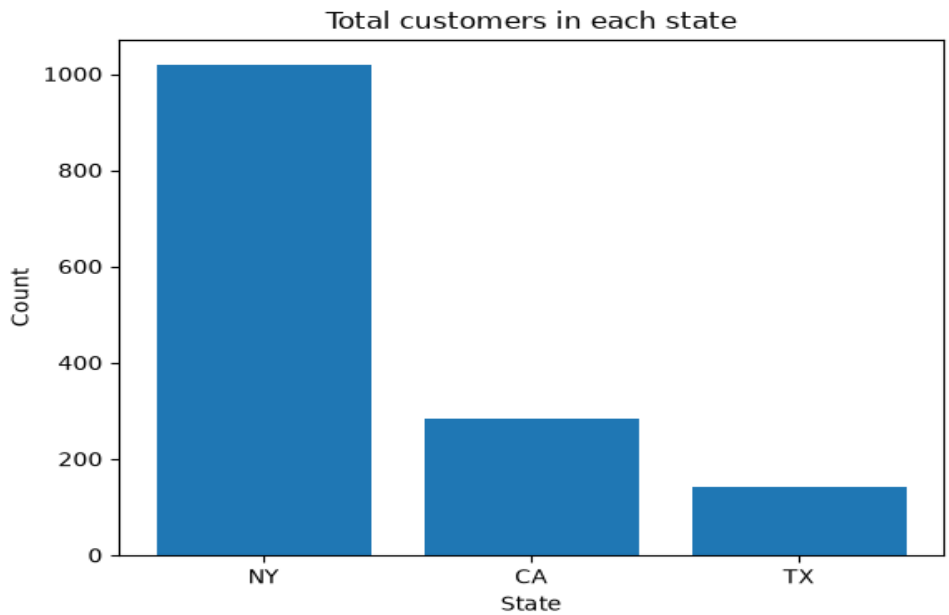


Figure 2: Total customers by state (bar chart).

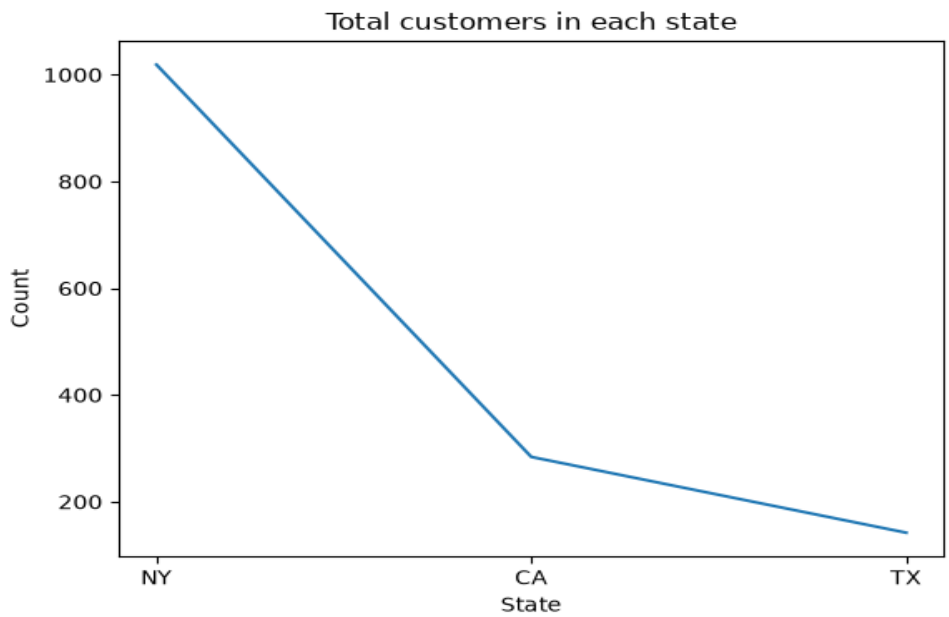


Figure 3: Total customers by state (trend view).

5.2 RFM Feature Engineering & Customer Segmentation

Recency, Frequency and Monetary (RFM) features were computed for every customer to support segmentation:

- Recency – number of days between each customer's most recent order and a fixed reference date (max order date + 1 day).

- Frequency – count of distinct orders placed by the customer.
- Monetary – total value of all purchases (sum of TotalSales) by the customer.

The orders and order_items tables were merged on OrderId and grouped by CustomerId to produce the RFM table:

```
[38]: consumer_segments = consumer_segments.reset_index()
      consumer_segments
```

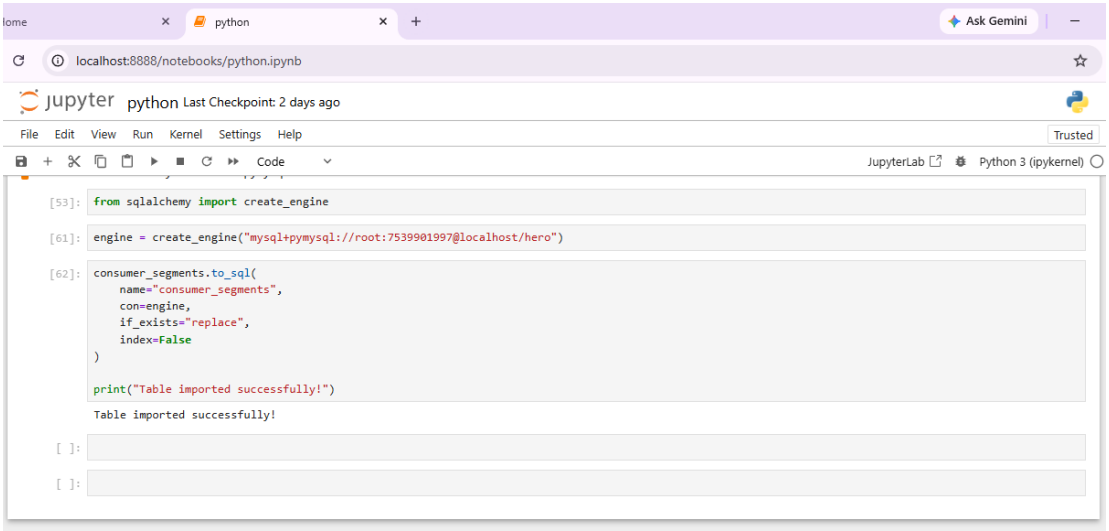
[38]:

	level_0	index	CustomerId	Recency	Frequency	Monetary
0	0	0	1	41	3	27888.1834
1	1	1	2	264	3	19329.0849
2	2	2	3	69	3	24051.5279
3	3	3	4	255	3	21150.8927
4	4	4	5	256	3	17520.2919
...	...	...	...	...	...	...
1440	1440	1440	1441	337	1	9173.3817
1441	1441	1441	1442	517	1	6987.2487
1442	1442	1442	1443	774	1	10372.0934
1443	1443	1443	1444	739	1	1399.9760
1444	1444	1444	1445	297	1	7999.9840

1445 rows × 6 columns

5.3 Exporting Results Back to SQL

The completed RFM/segmentation DataFrame was written back into the hero database as the consumer_segments table using SQLAlchemy's to_sql(), closing the loop between Python analysis and the SQL layer so Power BI could consume the segmentation results directly.

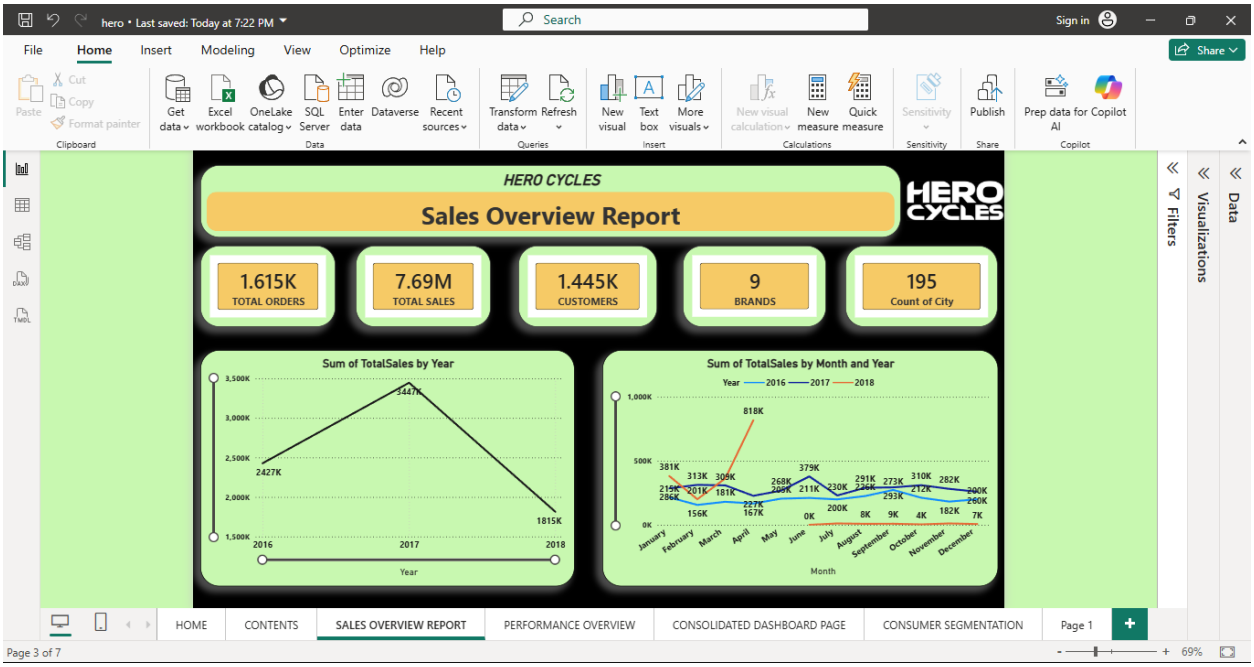


6. Phase 4 – Power BI: Visualization & Dashboarding

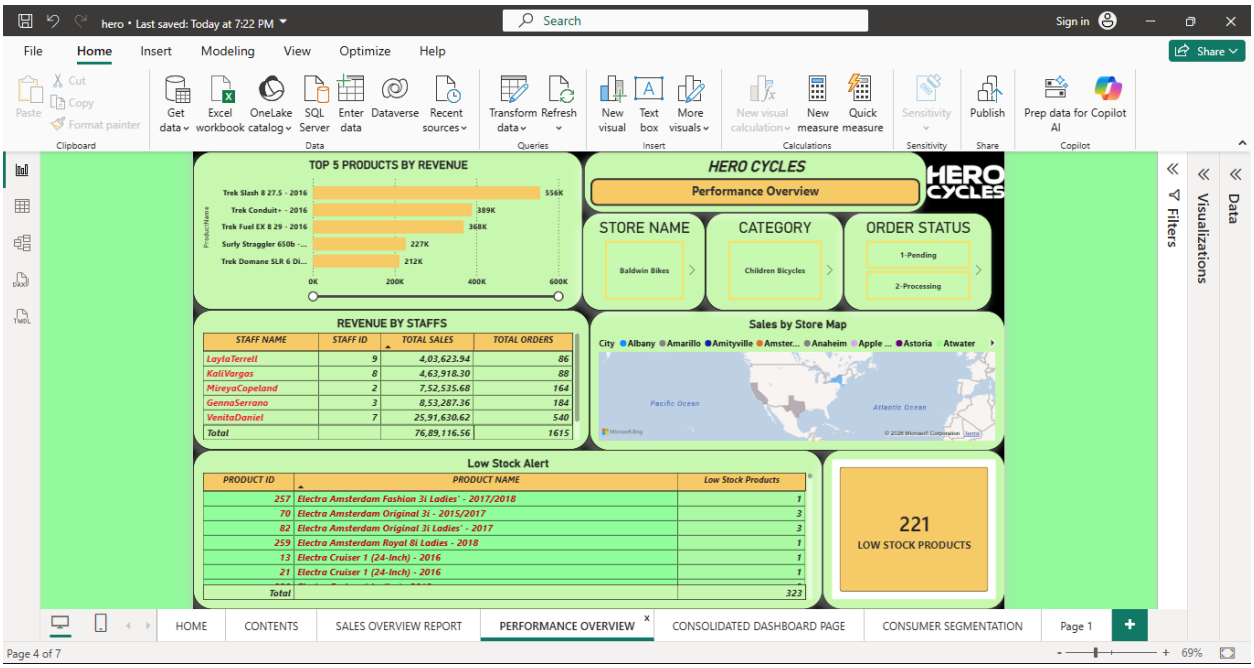
A Power BI report (hero.pbix) was built on top of the SQL database, modelling relationships between orders, order_items, products, customers, stores and staffs, and consuming the customer_segments table produced in Python. The report contains seven pages:

Page	Key Visuals
HOME	Landing / navigation page with entry button into the report.
CONTENTS	Navigation hub with action buttons to each report page.
Sales Overview Report	Sales trend (line chart), cumulative sales (area chart), and summary KPI cards for revenue, orders and related metrics.
Performance Overview	Top 5 Products by Revenue (bar chart), Sales by Store Map, Revenue by Staff (table), Low Stock Alert (table), with slicers for Category, Store Name and Order Status.
Consolidated Dashboard Page	Headline KPI cards, Customer Purchase Analysis (bar chart by city/state), and a report-wide slicer.
Consumer Segmentation	Distribution of Customer Segments (column chart), Segment share (donut chart), Total Revenue by segment (bar chart), with a Consumer Segment slicer.
Page 1	Supplementary KPI card page.

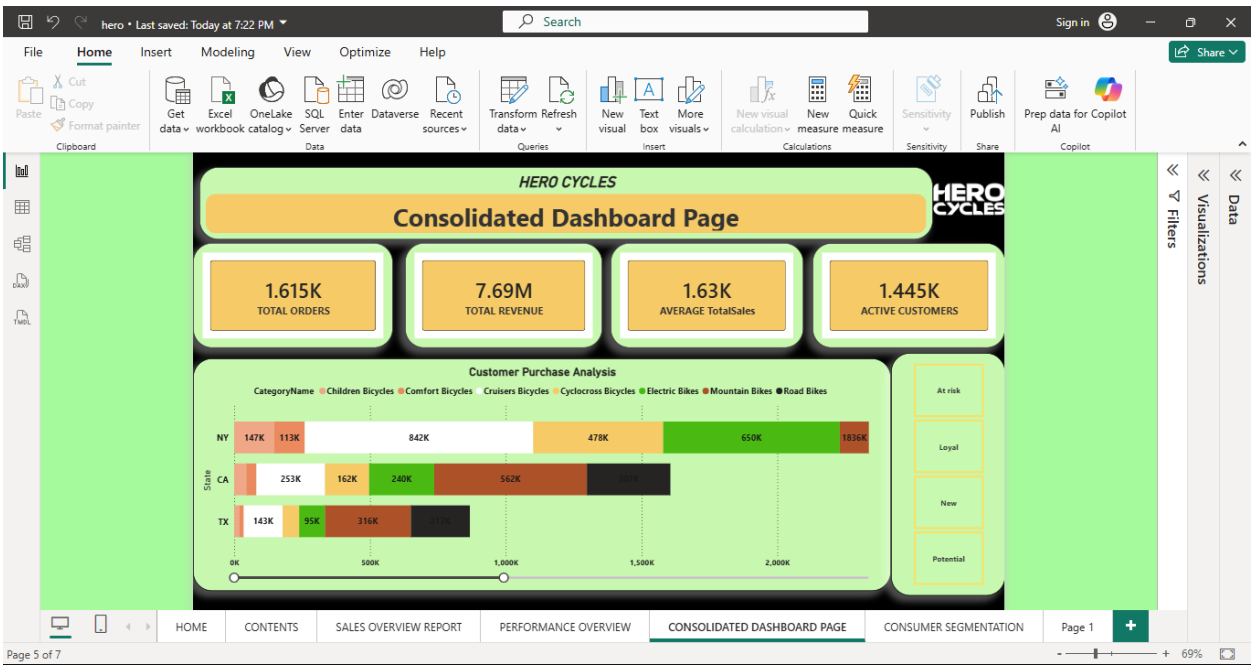
Sales Overview Report



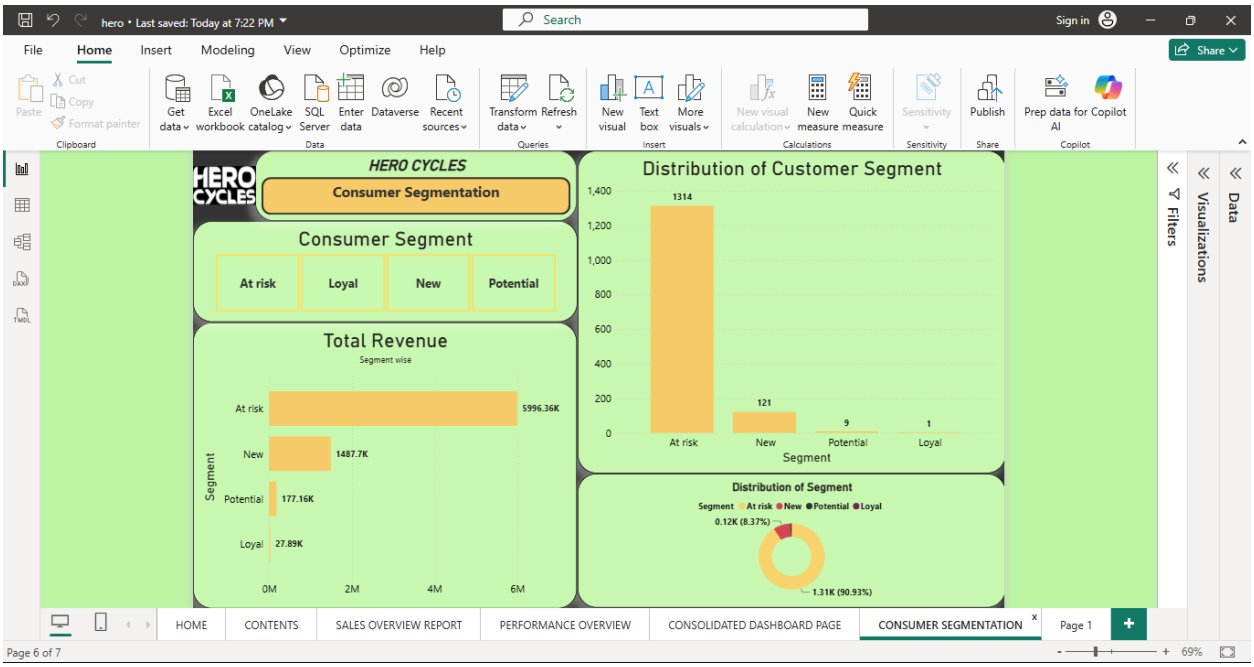
Performance Overview



Consolidated Dashboard Page



Consumer Segmentation:



6.1 Report Highlights

- Relationships modelled in Model View link orders.CustomerId → customers.CustomerId, orders.StoreId → stores.StoreId, orders.StaffId → staffs.StaffId, and order_items.ProductId → products.ProductId, matching the ERD.
- A geographic map visual plots sales by store location.
- Conditional formatting and a dedicated table highlight products with stock below the alert threshold.
- Interactive slicers for Order Status, Product Category and Store allow cross-filtering across visuals.
- The customer_segments output from Python is visualised with a donut/column chart and used as a report-wide filter, enabling segment-level revenue breakdowns.

7. Key Insights

- Sales, orders and revenue are concentrated across a small footprint of 3 stores and 10 staff members, making per-store and per-staff performance comparisons directly actionable.
- A defined set of top-5 products, identified both in SQL (by quantity) and Power BI (by revenue), account for a disproportionate share of sales — useful for inventory prioritisation.
- Customer spend is unevenly distributed: the SQL spending-bracket query and the Python RFM/segmentation model both support splitting the 1,445-customer base into low, medium and high-value groups for targeted marketing.
- The stock alert query surfaces products at or near low-inventory thresholds across the 939 store-product stock records, supporting proactive restocking.

- RFM segmentation gives a quantitative basis (Recency, Frequency, Monetary) for classifying customers into groups such as loyal, new and at-risk, which are then visualised and filterable in the Power BI Consumer Segmentation page.

8. Project Deliverables

Deliverable	File(s)
Cleaned Excel files	brands.xlsx, categories.xlsx, customers.xlsx, order_items.xlsx, orders.xlsx, products.xlsx, staffs.xlsx, stocks.xlsx, stores.xlsx
SQL scripts	hero.sql — database creation and all 8 analytical queries
Python notebook	python.ipynb — EDA, RFM calculation, and SQL export
Power BI report	hero.pbix — 7-page interactive dashboard

9. Conclusion

The Hero Retail Sales Analytics project delivers a complete ETL and analytics pipeline: raw data was cleaned in Excel, structured and queried in SQL, analysed and segmented using Python, and visualised through an interactive Power BI report. Together these outputs give management a repeatable way to monitor store and staff performance, track top products, manage stock levels, and target customers by value segment.